Python Lab Practice Set - Answer Key

Teacher copy: reference solutions and the main idea for every problem

All solutions were executed on the sample inputs and produce the printed sample outputs. Students may use different, equally valid approaches.

E1: Number Pyramid — EASY

Idea: Build each row as: leading spaces + ascending digits + descending digits.

```
n = int(input())
for i in range(1, n + 1):
    up = ''.join(str(j) for j in range(1, i + 1))
    down = ''.join(str(j) for j in range(i - 1, 0, -1))
    print(' ' * (n - i) + up + down)
```

E2: Alphabet Staircase — EASY

Idea: Keep a running counter of letters used; the letter is chr(65 + counter % 26).

```
n = int(input())
c = 0
for i in range(1, n + 1):
    print(''.join(chr(65 + (c + k) % 26) for k in range(i)))
    c += i
```

E3: Hollow Right Triangle — EASY

Idea: Rows 1 and N are solid; middle rows are '*' + (i-2) spaces + '*'.

```
n = int(input())
for i in range(1, n + 1):
    if i == 1 or i == n:
        print('*' * i)
    else:
        print('*' + ' ' * (i - 2) + '*')
```

E4: Star Hourglass — EASY

Idea: Let k = distance from the top or bottom edge; spaces = k, stars = 2*(N-1-k)+1.

```
n = int(input())
for i in range(2 * n - 1):
    k = i if i < n else 2 * n - 2 - i
    print(' ' * k + '*' * (2 * (n - 1 - k) + 1))
```

E5: Town Electricity Bill — EASY

Idea: Compute each slab with min/max, then apply surcharge and fixed charge.

```
cat = input().strip()
u = int(input())
if cat not in ('H', 'C') or u < 0:
    print("INVALID")
else:
    if cat == 'H':
        e = min(u, 100) * 3 + max(0, min(u, 200) - 100) * 5 + max(0, u - 200) * 8
        fixed = 50
    else:
        e = min(u, 200) * 9 + max(0, u - 200) * 12
        fixed = 150
    sur = e * 10 // 100 if e > 1000 else 0
    print(e + sur + fixed)
```

E6: Armstrong Number Hunt — EASY

Idea: For each number compute digit-power sum using len(str(n)).

```
a = int(input()); b = int(input())
res = []
for n in range(a, b + 1):
    s = str(n)
    if sum(int(d) ** len(s) for d in s) == n:
        res.append(n)
print(len(res))
print(*res if res else [-1])
```

E7: Caesar Cipher with Digits — EASY

Idea: Use modulo on the offset from 'A', 'a' or '0'. Python's % handles negative K correctly.

```
k = int(input())
s = input()
out = []
for ch in s:
    if 'A' <= ch <= 'Z':
        out.append(chr((ord(ch) - 65 + k) % 26 + 65))
    elif 'a' <= ch <= 'z':
        out.append(chr((ord(ch) - 97 + k) % 26 + 97))
    elif ch.isdigit():
        out.append(str((int(ch) + k) % 10))
    else:
        out.append(ch)
print(''.join(out))
```

E8: Move Zeroes — EASY

Idea: Keep a write pointer 'pos'; swap each non-zero element into it.

```
n = int(input())
a = list(map(int, input().split()))
pos = 0
for i in range(n):
    if a[i] != 0:
        a[pos], a[i] = a[i], a[pos]
        pos += 1
print(*a)
print(a.count(0))
```

E9: Bracket Checker — EASY

Idea: Stack of (bracket, position). Mismatch on closing -> that position; leftovers -> bottom of stack.

```
s = input()
pairs = {')': '(', ']': '[', '}': '{'}
st = []
bad = None
for i, ch in enumerate(s, 1):
    if ch in '([{':
        st.append((ch, i))
    elif ch in pairs:
        if not st or st[-1][0] != pairs[ch]:
            bad = i
            break
        st.pop()
if bad is None and st:
    bad = st[0][1]
print("VALID" if bad is None else "INVALID " + str(bad))
```

E10: Word Frequency Board

EASY

Idea: Dictionary counting, then sorted() with key=(-count, word).

```
t = input().lower()
for ch in '.,!?':
    t = t.replace(ch, '')
cnt = {}
for w in t.split():
    cnt[w] = cnt.get(w, 0) + 1
for w, c in sorted(cnt.items(), key=lambda x: (-x[1], x[0])):
    print(w, c)
```

E11: Nearest Point to Origin

EASY

Idea: min() with a key returns the first minimum, which matches the tie rule.

```
n = int(input())
pts = [tuple(map(int, input().split())) for _ in range(n)]
best = min(pts, key=lambda p: p[0] ** 2 + p[1] ** 2)
print(*best)
print(best[0] ** 2 + best[1] ** 2)
q = [0, 0, 0, 0]
for x, y in pts:
    if x > 0 and y > 0: q[0] += 1
    elif x < 0 and y > 0: q[1] += 1
    elif x < 0 and y < 0: q[2] += 1
    elif x > 0 and y < 0: q[3] += 1
print(*q)
```

E12: Club Membership Report

EASY

Idea: Union gives all students; count in how many sets each appears; intersection gives 'all three'.

```
clubs = []
for _ in range(3):
    k = int(input())
    clubs.append(set(input().split()))
allst = clubs[0] | clubs[1] | clubs[2]
cnt = {s: sum(s in c for c in clubs) for s in allst}
print(sum(1 for v in cnt.values() if v == 1))
print(sum(1 for v in cnt.values() if v == 2))
print(sum(1 for v in cnt.values() if v == 3))
print(len(allst))
both = sorted(clubs[0] & clubs[1] & clubs[2])
print(*both if both else ["NONE"])
```

E13: Stairs with Broken Steps

EASY

Idea: ways[i] = ways[i-1] + ways[i-2], but 0 if step i is broken.

```
n = int(input()); k = int(input())
broken = set(map(int, input().split()))
ways = [0] * (n + 1)
ways[0] = 1
for i in range(1, n + 1):
    if i in broken:
        continue
    ways[i] = ways[i - 1] + (ways[i - 2] if i >= 2 else 0)
print(ways[n])
```

E14: Collatz Champion

EASY

Idea: Simulate each start; update the best only on strictly more steps so ties keep the smaller number.

```
n = int(input())
best = (-1, 0, 0)
for s in range(1, n + 1):
    x = s; steps = 0; hi = s
    while x != 1:
        x = x // 2 if x % 2 == 0 else 3 * x + 1
        steps += 1
        hi = max(hi, x)
    if steps > best[1] or best[0] == -1:
        best = (s, steps, hi)
print(best[0]); print(best[1]); print(best[2])
```

E15: Shop Receipt Formatter

EASY

Idea: Use f-string format specs: <10 (left), >4 (right) and >8.2f (right with 2 decimals).

```
n = int(input())
print(f"{'ITEM':<10}{'QTY':>4}{'PRICE':>8}{'AMOUNT':>9}")
print('-' * 31)
total = 0.0
for _ in range(n):
    name, q, p = input().split()
    q = int(q); p = float(p)
    amt = q * p
    total += amt
    print(f"{name:<10}{q:>4}{p:>8.2f}{amt:>9.2f}")
print('-' * 31)
gst = total * 0.05
print(f"{'TOTAL':<22}{total:>9.2f}")
print(f"{'GST(5%)':<22}{gst:>9.2f}")
print(f"{'PAYABLE':<22}{total + gst:>9.2f}")
```

E16: Roll Call Pairs

EASY

Idea: zip() builds (name, mark) tuples; max() with key returns the first maximum.

```
n = int(input())
names = input().split()
marks = list(map(int, input().split()))
pairs = list(zip(names, marks))
top = max(pairs, key=lambda p: p[1])
avg = sum(marks) / n
above = [nm for nm, m in pairs if m > avg]
print(*top)
print("{:.2f}".format(avg))
print(len(above))
print(*above if above else ["NONE"])
```

E17: Scholarship Eligibility

EASY

Idea: Validate first, then an if-elif ladder in the order given.

```
m = int(input()); a = int(input()); i = int(input()); b = int(input())
if not (0 <= m <= 100 and 0 <= a <= 100 and b in (0, 1) and i >= 0):
    print("INVALID")
elif m >= 90 and a >= 75 and b == 0:
    print("FULL")
elif m >= 75 and a >= 75 and b == 0 and i <= 30000:
    print("HALF")
elif m >= 60 and a >= 65 and i <= 15000:
    print("QUARTER")
else:
    print("NONE")
```

M1: Hollow Diamond — MEDIUM

Idea: Let k = distance of the row from the top/bottom half; leading spaces = N-1-k and gap = 2k-1.

```
n = int(input())
for i in range(2 * n - 1):
    k = i if i < n else 2 * n - 2 - i
    sp = ' ' * (n - 1 - k)
    if k == 0:
        print(sp + '*')
    else:
        print(sp + '*' + ' ' * (2 * k - 1) + '*')
```

M2: Spiral Matrix Builder — MEDIUM

Idea: Shrink four boundaries (top, bottom, left, right) after filling each side.

```
n = int(input())
g = [[0] * n for _ in range(n)]
top, bot, left, right = 0, n - 1, 0, n - 1
v = 1
while top <= bot and left <= right:
    for j in range(left, right + 1): g[top][j] = v; v += 1
    top += 1
    for i in range(top, bot + 1): g[i][right] = v; v += 1
    right -= 1
    if top <= bot:
        for j in range(right, left - 1, -1): g[bot][j] = v; v += 1
        bot -= 1
    if left <= right:
        for i in range(bot, top - 1, -1): g[i][left] = v; v += 1
        left += 1
w = len(str(n * n))
for row in g:
    print(' '.join(str(x).rjust(w) for x in row))
```

M3: Butterfly Pattern — MEDIUM

Idea: Write a helper that builds one row; call it for 1..N then N-1..1.

```
n = int(input())
def row(i):
    return '*' * i + ' ' * (2 * (n - i)) + '*' * i
for i in range(1, n + 1):
    print(row(i))
for i in range(n - 1, 0, -1):
    print(row(i))
```

M4: X Mark — MEDIUM

Idea: A cell is a star when j == i or j == N-1-i. Use rstrip() to remove trailing spaces.

```
n = int(input())
for i in range(n):
    row = ''
    for j in range(n):
        row += '*' if j == i or j == n - 1 - i else ' '
    print(row.rstrip())
```

M5: Parking Lot Revenue — MEDIUM

Idea: Convert times to minutes, ceil with (d+59)//60, apply cap; track best fee with strict > for first-wins ties.

```
def mins(t):
    h, m = t.split(':')
    return int(h) * 60 + int(m)
n = int(input())
total = 0; free = 0
best = None
for _ in range(n):
    plate, a, b = input().split()
    d = mins(b) - mins(a)
    if d <= 10:
        fee = 0; free += 1
    else:
        hrs = (d + 59) // 60
        fee = min(150, 20 + 15 * (hrs - 1))
    total += fee
    if best is None or fee > best[1]:
        best = (plate, fee)
print(total)
print(*best)
print(free)
```

M6: Library Fine Calculator — MEDIUM

Idea: Dictionary id -> issue day. pop() on return and compute lateness.

```
n = int(input())
out = {}
fine = 0
for _ in range(n):
    op, bid, day = input().split(); day = int(day)
    if op == 'ISSUE':
        out[bid] = day
    else:
        late = day - out.pop(bid) - 7
        if late > 0:
            fine += 2 * late
print(fine)
print(len(out))
print(*sorted(out) if out else ["NONE"])
```

M7: Weekly Attendance Report — MEDIUM

Idea: Count 'P' for percent; track a running streak of 'A' characters.

```
n, d = map(int, input().split())
defaulters = []
bestname, bestrun = None, 0
for _ in range(n):
    name, s = input().split()
    pct = s.count('P') * 100 / d
    print(name, "{:.1f}".format(pct))
    if pct < 75:
        defaulters.append(name)
    run = mx = 0
    for ch in s:
        run = run + 1 if ch == 'A' else 0
        mx = max(mx, run)
    if mx > bestrun:
        bestname, bestrun = name, mx
print("Defaulters:", *(defaulters if defaulters else ["NONE"]))
print("Longest absence:", *((bestname, bestrun) if bestrun else ["NONE"]))
```

M8: Rank List with Ties — MEDIUM

Idea: Sort with key (-marks, name). The rank is the index+1 whenever the marks change.

```
n = int(input())
st = []
for _ in range(n):
    nm, m = input().split()
    st.append((nm, int(m)))
st.sort(key=lambda x: (-x[1], x[0]))
rank = 0
for i, (nm, m) in enumerate(st):
    if i == 0 or m != st[i - 1][1]:
        rank = i + 1
    print(rank, nm, m)
```

M9: Longest Unique Stretch — MEDIUM

Idea: Sliding window; a dictionary stores the last index of each character. Move 'start' past a repeat.

```
s = input().strip()
last = {}
start = 0
bl, bs = 0, 0
for i, ch in enumerate(s):
    if ch in last and last[ch] >= start:
        start = last[ch] + 1
    last[ch] = i
    if i - start + 1 > bl:
        bl, bs = i - start + 1, start
print(bl)
print(s[bs:bs + bl])
```

M10: Prime Gap Finder — MEDIUM

Idea: Sieve of Eratosthenes, then scan consecutive primes with zip(pr, pr[1:]).

```
a = int(input()); b = int(input())
sieve = [True] * (b + 1)
sieve[0] = False
if b >= 1: sieve[1] = False
i = 2
while i * i <= b:
    if sieve[i]:
        for j in range(i * i, b + 1, i):
            sieve[j] = False
    i += 1
pr = [x for x in range(a, b + 1) if sieve[x]]
print(len(pr))
if len(pr) < 2:
    print(-1)
else:
    g, p, q = 0, 0, 0
    for x, y in zip(pr, pr[1:]):
        if y - x > g:
            g, p, q = y - x, x, y
    print(g, p, q)
```

M11: Magic Square Inspector — MEDIUM

Idea: Check rules in order using sorted(flat) == range, row sums, column sums via comprehension, and two diagonals.

```
n = int(input())
g = [list(map(int, input().split())) for _ in range(n)]
flat = [x for r in g for x in r]
target = sum(g[0])
if sorted(flat) != list(range(1, n * n + 1)):
    print("NO"); print("DISTINCT")
elif any(sum(r) != target for r in g):
    print("NO"); print("ROWS")
elif any(sum(g[i][j] for i in range(n)) != target for j in range(n)):
    print("NO"); print("COLUMNS")
elif sum(g[i][i] for i in range(n)) != target or sum(g[i][n - 1 - i] for i in range(n)) != target:
    print("NO"); print("DIAGONALS")
else:
    print("YES", target)
```

M12: Run-Length Compression — MEDIUM

Idea: Two-pointer grouping of runs; build the encoding and compare lengths.

```
s = input().strip()
enc = []
i = 0
while i < len(s):
    j = i
    while j < len(s) and s[j] == s[i]:
        j += 1
    enc.append(s[i] + (str(j - i) if j - i > 1 else ''))
    i = j
e = ''.join(enc)
if len(e) < len(s):
    print(e); print("COMPRESSED")
else:
    print(s); print("ORIGINAL")
```

M13: Password Strength Auditor — MEDIUM

Idea: One function returning the number of rules passed; booleans add as 0/1.

```
def score(p):
    s = 0
    s += len(p) >= 8
    s += any(c.isupper() for c in p)
    s += any(c.islower() for c in p)
    s += any(c.isdigit() for c in p)
    s += any(c in '@#$%&*!' for c in p)
    s += not any(p[i] == p[i + 1] == p[i + 2] for i in range(len(p) - 2))
    s += 'password' not in p.lower()
    return s
n = int(input())
c = {'STRONG': 0, 'MEDIUM': 0, 'WEAK': 0}
for _ in range(n):
    sc = score(input().strip())
    cat = 'STRONG' if sc == 7 else ('MEDIUM' if sc >= 5 else 'WEAK')
    c[cat] += 1
    print(cat, sc)
print("STRONG=%d MEDIUM=%d WEAK=%d" % (c['STRONG'], c['MEDIUM'], c['WEAK']))
```

M14: Spreadsheet Column Names

MEDIUM

Idea: Bijective base-26: subtract 1 before taking % 26 each time.

```
n = int(input()); t = input().strip()
name = ''
m = n
while m > 0:
    m -= 1
    name = chr(65 + m % 26) + name
    m //= 26
num = 0
for ch in t:
    num = num * 26 + (ord(ch) - 64)
print(name); print(num)
```

M15: Anagram Clusters

MEDIUM

Idea: Key = sorted letters of the word. Group words in a dictionary, filter groups of size >= 2.

```
n = int(input())
words = input().split()
g = {}
for w in words:
    g.setdefault(''.join(sorted(w)), []).append(w)
cl = [sorted(v) for v in g.values() if len(v) >= 2]
cl.sort(key=lambda c: (-len(c), c[0]))
print(len(cl))
for c in cl:
    print(*c)
```

M16: Friend Suggestions

MEDIUM

Idea: Dictionary of sets. Count how often each friend-of-friend appears, skipping the target and existing friends.

```
n = int(input())
g = {}
for _ in range(n):
    p = input().split()
    g[p[0]] = set(p[2:])
t = input().strip()
cnt = {}
for f in g[t]:
    for ff in g[f]:
        if ff != t and ff not in g[t]:
            cnt[ff] = cnt.get(ff, 0) + 1
if not cnt:
    print("NONE")
for nm, c in sorted(cnt.items(), key=lambda x: (-x[1], x[0])):
    print(nm, c)
```

M17: Operation Pipeline

MEDIUM

Idea: A dictionary maps the operation name to a function; apply it with a list comprehension.

```
n = int(input())
a = list(map(int, input().split()))
k = int(input())
ops = {
    'ADD': lambda v, x: v + x,
    'MUL': lambda v, x: v * x,
    'MOD': lambda v, x: v % x,
    'SQUARE': lambda v, x: v * v,
    'NEG': lambda v, x: -v,
    'ABS': lambda v, x: abs(v),
}
for _ in range(k):
    p = input().split()
    x = int(p[1]) if len(p) > 1 else 0
    a = [ops[p[0]](v, x) for v in a]
print(*a)
print(sum(a))
```

H1: Concentric Square HARD

Idea: Value at (i, j) = max(|i-c|, |j-c|) + 1 where c is the centre index.

```
n = int(input())
size = 2 * n - 1
c = n - 1
for i in range(size):
    print(''.join(str(max(abs(i - c), abs(j - c)) + 1) for j in range(size)))
```

H2: Centered Pascal Triangle HARD

Idea: Build rows from the previous one, convert to strings, and compute padding from the last row's length.

```
n = int(input())
rows = [[1]]
for _ in range(n - 1):
    p = rows[-1]
    rows.append([1] + [p[i] + p[i + 1] for i in range(len(p) - 1)] + [1])
strs = [' '.join(map(str, r)) for r in rows]
w = len(strs[-1])
for s in strs:
    print(' ' * ((w - len(s)) // 2) + s)
```

H3: Star Wave HARD

Idea: The row of column j follows a triangle wave: pos = j % (2(N-1)); row = pos if pos < N else 2(N-1)-pos.

```
n = int(input()); w = int(input())
per = 2 * (n - 1)
grid = [[' '] * w for _ in range(n)]
for j in range(w):
    pos = j % per
    r = pos if pos < n else per - pos
    grid[r][j] = '*'
for row in grid:
    print(''.join(row).rstrip())
```

H4: Hollow Hourglass HARD

Idea: Same k as the solid hourglass; three cases: edge row, middle row, and hollow rows with gap 2(N-1-k)-1.

```
n = int(input())
for i in range(2 * n - 1):
    k = i if i < n else 2 * n - 2 - i
    if k == 0:
        print('*' * (2 * n - 1))
    elif k == n - 1:
        print(' ' * k + '*')
    else:
        print(' ' * k + '*' + ' ' * (2 * (n - 1 - k) - 1) + '*')
```

H5: Trapped Rain Water HARD

Idea: Water above bar i = min(max height on left, max height on right) - h[i].

```
n = int(input())
h = list(map(int, input().split()))
left = [0] * n; right = [0] * n
m = 0
for i in range(n):
    m = max(m, h[i]); left[i] = m
m = 0
for i in range(n - 1, -1, -1):
    m = max(m, h[i]); right[i] = m
water = [min(left[i], right[i]) - h[i] for i in range(n)]
print(sum(water))
print(max(water))
```

H6: Cinema Group Booking HARD

Idea: Scan rows top to bottom and start positions left to right; take the first window of k free seats.

```
r, c = map(int, input().split())
q = int(input())
g = [[0] * c for _ in range(r)]
booked = 0
for _ in range(q):
    k = int(input())
    done = False
    for i in range(r):
        for s in range(c - k + 1):
            if all(g[i][s + t] == 0 for t in range(k)):
                for t in range(k):
                    g[i][s + t] = 1
                print(i + 1, "%d-%d" % (s + 1, s + k))
                booked += k
                done = True
                break
        if done:
            break
    if not done:
        print("REJECTED")
print(booked)
print(sum(1 for row in g if all(row)))
```

H7: Merging Booked Slots HARD

Idea: Sort by start; extend the last block if the new interval starts within it.

```
n = int(input())
iv = sorted(tuple(map(int, input().split())) for _ in range(n))
res = []
for a, b in iv:
    if res and a <= res[-1][1]:
        res[-1][1] = max(res[-1][1], b)
    else:
        res.append([a, b])
print(len(res))
for a, b in res:
    print(a, b)
print(sum(b - a for a, b in res))
```

H8: Subarray Sum Detective HARD

Idea: Prefix sums: a subarray ends at i with sum K if prefix[i]-K was seen before. Keep counts and first indices.

```
n, k = map(int, input().split())
a = list(map(int, input().split()))
cnt = {0: 1}
first = {0: -1}
s = 0; total = 0; longest = 0
for i, x in enumerate(a):
    s += x
    total += cnt.get(s - k, 0)
    if s - k in first:
        longest = max(longest, i - first[s - k])
    cnt[s] = cnt.get(s, 0) + 1
    first.setdefault(s, i)
print(total)
print(longest)
```

H9: Warmer Days Ahead — HARD

Idea: Monotonic stack of indices with decreasing temperatures; pop when a warmer day appears.

```
n = int(input())
t = list(map(int, input().split()))
ans = [0] * n
st = []
for i in range(n):
    while st and t[st[-1]] < t[i]:
        j = st.pop()
        ans[j] = i - j
    st.append(i)
print(*ans)
m = max(ans)
print(m, ans.index(m) + 1 if m > 0 else -1)
```

H10: Coin Change Planner — HARD

Idea: Two DP arrays: min coins, and combinations (loop coins outermost so order is ignored).

```
n, t = map(int, input().split())
coins = list(map(int, input().split()))
INF = float('inf')
mn = [0] + [INF] * t
for x in range(1, t + 1):
    for c in coins:
        if c <= x and mn[x - c] + 1 < mn[x]:
            mn[x] = mn[x - c] + 1
ways = [1] + [0] * t
for c in coins:
    for x in range(c, t + 1):
        ways[x] += ways[x - c]
print(mn[t] if mn[t] != INF else -1)
print(ways[t])
```

H11: League Table Builder — HARD

Idea: Dictionary of dictionaries for stats; sorted() with a 4-part key tuple.

```
m = int(input())
T = {}
def rec(n):
    return T.setdefault(n, {'p': 0, 'w': 0, 'd': 0, 'l': 0, 'gf': 0, 'ga': 0})
for _ in range(m):
    a, ga, _, gb, b = input().split()
    ga, gb = int(ga), int(gb)
    x, y = rec(a), rec(b)
    x['p'] += 1; y['p'] += 1
    x['gf'] += ga; x['ga'] += gb; y['gf'] += gb; y['ga'] += ga
    if ga > gb: x['w'] += 1; y['l'] += 1
    elif ga < gb: y['w'] += 1; x['l'] += 1
    else: x['d'] += 1; y['d'] += 1
def pts(r): return 3 * r['w'] + r['d']
order = sorted(T, key=lambda n: (-pts(T[n]), -(T[n]['gf'] - T[n]['ga']), -T[n]['gf'], n))
for i, n in enumerate(order, 1):
    r = T[n]
    print(i, n, r['p'], r['w'], r['d'], r['l'], r['gf'], r['ga'], r['gf'] - r['ga'], pts(r))
```

H12: Word Hunt Grid — HARD

Idea: Backtracking DFS from every start cell with a visited set that is undone on return.

```
r, c = map(int, input().split())
g = [input().strip() for _ in range(r)]
w = input().strip()
def dfs(i, j, k, seen):
    if g[i][j] != w[k]:
        return False
    if k == len(w) - 1:
        return True
    seen.add((i, j))
    for di, dj in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        ni, nj = i + di, j + dj
        if 0 <= ni < r and 0 <= nj < c and (ni, nj) not in seen:
            if dfs(ni, nj, k + 1, seen):
                seen.discard((i, j))
                return True
    seen.discard((i, j))
    return False
cnt = sum(1 for i in range(r) for j in range(c) if dfs(i, j, 0, set()))
print("YES", cnt) if cnt else print("NO")
```

H13: Mini Bank (Classes) — HARD

Idea: Account class with deposit/withdraw raising a custom exception; main loop catches it and prints the reason.

```
class BankError(Exception):
    def __init__(self, reason):
        self.reason = reason

class Account:
    MIN = 500
    def __init__(self, name, bal):
        if bal < self.MIN:
            raise BankError("INVALID_AMOUNT")
        self.name, self.balance = name, bal
    def deposit(self, x):
        if x <= 0:
            raise BankError("INVALID_AMOUNT")
        self.balance += x
    def withdraw(self, x):
        if x <= 0:
            raise BankError("INVALID_AMOUNT")
        if self.balance - x < self.MIN:
            raise BankError("INSUFFICIENT")
        self.balance -= x

acc = {}
n = int(input())
for k in range(1, n + 1):
    p = input().split()
    try:
        if p[0] == 'CREATE':
            if p[1] in acc: raise BankError("DUPLICATE")
            acc[p[1]] = Account(p[1], int(p[2]))
        elif p[0] == 'DEPOSIT':
            if p[1] not in acc: raise BankError("NO_ACCOUNT")
            acc[p[1]].deposit(int(p[2]))
        elif p[0] == 'WITHDRAW':
            if p[1] not in acc: raise BankError("NO_ACCOUNT")
            acc[p[1]].withdraw(int(p[2]))
        else:
            if p[1] not in acc or p[2] not in acc: raise BankError("NO_ACCOUNT")
            amt = int(p[3])
            acc[p[1]].withdraw(amt)
            acc[p[2]].deposit(amt)
    except BankError as e:
        print("ERROR", k, e.reason)
for name in sorted(acc):
    print(name, acc[name].balance)
```

H14: Island Survey HARD

Idea: Flood fill from each unvisited land cell, tracking area and whether a border cell was touched.

```
r, c = map(int, input().split())
g = [list(input().strip()) for _ in range(r)]
seen = [[False] * c for _ in range(r)]
cnt = best = border = 0
for i in range(r):
    for j in range(c):
        if g[i][j] == '1' and not seen[i][j]:
            cnt += 1
            stack = [(i, j)]; seen[i][j] = True
            area = 0; touch = False
            while stack:
                x, y = stack.pop()
                area += 1
                if x in (0, r - 1) or y in (0, c - 1):
                    touch = True
                for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < r and 0 <= ny < c and g[nx][ny] == '1' and not seen[nx][ny]:
                        seen[nx][ny] = True
                        stack.append((nx, ny))
            best = max(best, area)
            border += touch
print(cnt); print(best); print(border)
```

H15: Sudoku Validator HARD

Idea: Helper ok(cells) compares list length with set length; box b starts at row (b//3)*3, column (b%3)*3.

```
g = [input().strip() for _ in range(9)]
def ok(cells):
    d = [c for c in cells if c != '.']
    return len(d) == len(set(d))
ans = "VALID"
for r in range(9):
    if not ok(g[r]):
        ans = "INVALID ROW %d" % (r + 1); break
if ans == "VALID":
    for c in range(9):
        if not ok([g[r][c] for r in range(9)]):
            ans = "INVALID COL %d" % (c + 1); break
if ans == "VALID":
    for b in range(9):
        r0, c0 = (b // 3) * 3, (b % 3) * 3
        if not ok([g[r0 + i][c0 + j] for i in range(3) for j in range(3)]):
            ans = "INVALID BOX %d" % (b + 1); break
print(ans)
```

H16: Conference Rooms Needed HARD

Idea: Sweep line over sorted events; at equal times ends (-1) sort before starts (+1).

```
n = int(input())
ev = []
for _ in range(n):
    s, e = map(int, input().split())
    ev.append((s, 1))
    ev.append((e, -1))
ev.sort()
cur = best = t = 0
for time, d in ev:
    cur += d
    if cur > best:
        best, t = cur, time
print(best)
print(t)
```